Universidad de los Llanos · Ingeniería de Sistemas · Bases de Datos 2026-II

Proyecto #13: Sistema de Ventas por Internet

Juan Esteban Ramos Triviño 160005465
Andrés Jerónimo Puerta López 160005490
David Santiago Díaz Riveros 160005319

Docente: Jesús Reyes Carvajal · Noviembre de 2026

1. Objetivos

Objetivo general

Desarrollar un sistema web de ventas por Internet, con una base de datos en PostgreSQL y una aplicación en PHP sin frameworks sobre Linux, que permita registrar, actualizar y consultar los artículos, clientes, ciudades, ventas y pagos de una tienda, y hacer el seguimiento de cada venta desde el pedido hasta la entrega.

Objetivos específicos

- Diseñar el modelo entidad-relación y el modelo relacional de la tienda, normalizados hasta la tercera forma normal (3FN).
- Crear la base de datos con llaves primarias y foráneas, restricciones `CHECK` para que precios, cantidades, inventario y montos no sean negativos, y disparadores que controlen el inventario y las fechas de pago.
- Construir un panel de administración para gestionar artículos, categorías, clientes, ciudades, ventas y pagos, y una tienda donde los clientes registrados compran con un carrito y corrigen sus propios datos.
- Hacer el seguimiento de cada venta por sus estados (Pendiente, Pagada, Enviada, Entregada o Cancelada) y registrar sus pagos por PSE, tarjeta Visa o Mastercard, Efecty o transferencia bancaria.
- Validar los datos de entrada en la aplicación y en la base, y proteger la información con cuentas, roles y seguridad por filas, para que cada cliente vea solo sus compras.
- Publicar el sistema en Railway para usarlo desde cualquier navegador.

2. Justificación

Una tienda que vende por Internet maneja cada día pedidos, inventario y pagos de clientes de distintas ciudades. Si esa información se lleva a mano o en hojas de cálculo, aparecen errores comunes: vender un artículo que ya se agotó, perder el rastro de un pedido, registrar un pago con una fecha imposible o no saber cuánto le falta pagar a un cliente.

Este sistema guarda todo en una base de datos relacional que hace cumplir las reglas del negocio por sí misma: el inventario se descuenta solo al vender y nunca queda negativo, un pago no puede tener fecha anterior a su venta, no se puede borrar un cliente o un artículo que ya tiene ventas, y el total de cada venta se calcula siempre a partir de sus artículos, así que nunca queda desactualizado.

Además, el cliente compra desde la tienda y consulta el estado de sus pedidos sin intermediarios, y el administrador ve en un solo lugar qué ventas faltan por pagar, enviar o entregar. Como usa solo herramientas libres (PostgreSQL, PHP y Linux) y está publicado en la web, se puede usar desde cualquier lugar sin pagar licencias.

3. Modelo entidad-relación

El modelo tiene 7 entidades y 6 relaciones. Dentro de cada entidad están sus atributos, con la llave primaria subrayada. Imagen es una entidad débil: no tiene llave propia y solo existe si existe su artículo.

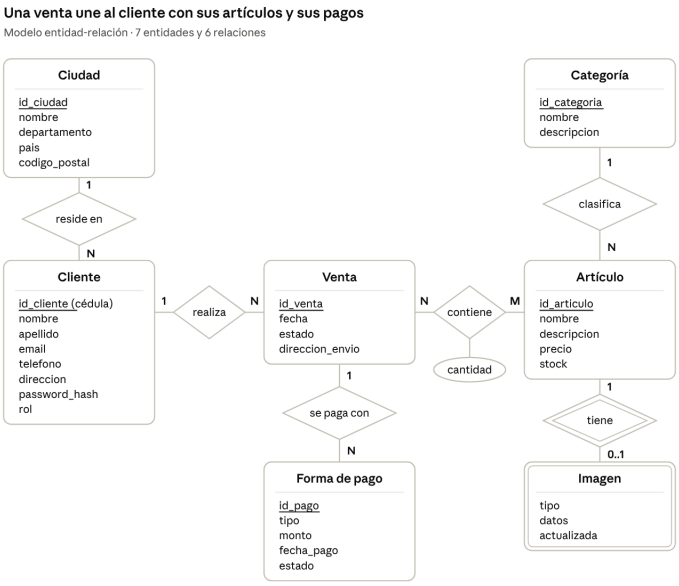


Figura 1. Modelo entidad-relación

Relación	Entidades	Cardinalidad	Qué significa
reside en	Cliente – Ciudad	N:1	Cada cliente vive en una ciudad; en una ciudad viven muchos clientes.
realiza	Cliente – Venta	1:N	Un cliente hace muchas compras; cada venta es de un solo cliente.
clasifica	Categoría – Artículo	1:N	Una categoría agrupa muchos artículos; cada artículo está en una sola categoría.
contiene	Venta – Artículo	N:M	Una venta lleva varios artículos y un artículo aparece en muchas ventas. La relación tiene su propio atributo, `cantidad`.
se paga con	Venta – Forma de pago	1:N	Una venta puede recibir varios pagos (por ejemplo, un abono y el resto); cada pago es de una sola venta.
tiene	Artículo – Imagen	1:0..1	Un artículo tiene como máximo una foto, y la foto pertenece a un solo artículo.

4. Modelo relacional

La base tiene 8 tablas unidas por 7 llaves foráneas. Salen de las 6 entidades del modelo entidad-relación, más la tabla puente `detalle_venta`, que resuelve la relación muchos a muchos entre ventas y artículos, y la tabla `articulo_imagen`, con la foto de cada artículo.

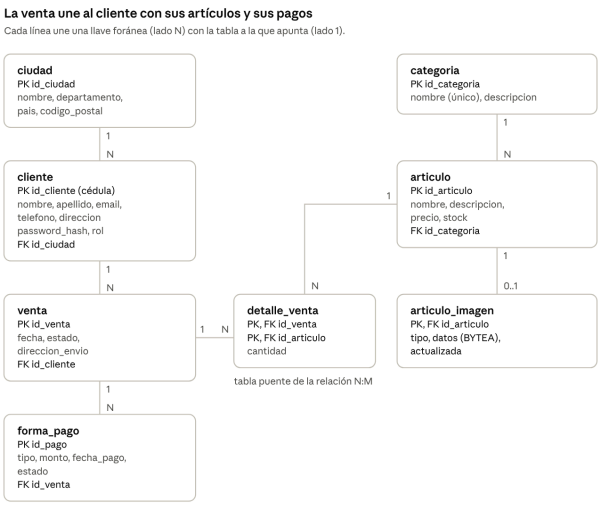


Figura 2. Modelo relacional

`venta` está en el centro: cada venta pertenece a un cliente, tiene una o más líneas de detalle (cada una con un artículo) y recibe uno o más pagos. Cada artículo tiene cero o una foto (relación 1 a 0..1), y `cliente` guarda también los datos de la cuenta: `password_hash` y `rol`.

Así se pasó del modelo entidad-relación a las tablas:

- Cada entidad es una tabla, con la misma llave primaria.

- Las cuatro relaciones 1:N se vuelven una llave foránea en la tabla del lado N: `cliente.id_ciudad`, `venta.id_cliente`, `articulo.id_categoria` y `forma_pago.id_venta`.
- La relación N:M *contiene* se vuelve la tabla `detalle_venta`, con llave primaria compuesta (`id_venta`, `id_articulo`) y el atributo `cantidad`.
- La entidad débil Imagen se vuelve `articulo_imagen`. Su llave primaria `id_articulo` es también llave foránea hacia `articulo`, y por eso cada artículo tiene como máximo una foto.

Diseño lógico

Llave primaria subrayada; llave foránea en cursiva, con la tabla a la que apunta.

- **CIUDAD** (id_ciudad, nombre, departamento, pais, codigo_postal)
- **CLIENTE** (id_cliente, nombre, apellido, email, telefono, direccion, password_hash, rol, *id_ciudad* → CIUDAD)
- **CATEGORIA** (id_categoria, nombre, descripcion)
- **ARTICULO** (id_articulo, nombre, descripcion, precio, stock, *id_categoria* → CATEGORIA)
- **ARTICULO_IMAGEN** (*id_articulo* → ARTICULO, tipo, datos, actualizada)
- **VENTA** (id_venta, fecha, estado, direccion_envio, *id_cliente* → CLIENTE)
- **DETALLE_VENTA** (*id_venta* → VENTA, *id_articulo* → ARTICULO, cantidad)
- **FORMA_PAGO** (id_pago, tipo, monto, fecha_pago, estado, *id_venta* → VENTA)

5. Diccionario de datos

Para cada tabla: el nombre de cada campo, su tipo en PostgreSQL, si acepta valores nulos, si es llave y qué guarda.

ciudad

Campo	Tipo	Nulo	Llave	Descripción
`id_ciudad`	SERIAL	No	PK	Número de la ciudad; lo asigna la base.
`nombre`	VARCHAR(80)	No	—	Nombre de la ciudad.
`departamento`	VARCHAR(80)	No	—	Departamento al que pertenece.
`pais`	VARCHAR(60)	No	—	País; por defecto 'Colombia'.
`codigo_postal`	VARCHAR(15)	No	—	Código postal.

cliente

Campo	Tipo	Nulo	Llave	Descripción
`id_cliente`	INT4	No	PK	Cédula del cliente; mayor que 0.
`nombre`	VARCHAR(80)	No	—	Nombres.
`apellido`	VARCHAR(80)	No	—	Apellidos.
`email`	VARCHAR(120)	No	UNIQUE	Correo; es también el usuario para iniciar sesión.
`telefono`	VARCHAR(20)	No	—	Teléfono de contacto.
`direccion`	VARCHAR(150)	No	—	Dirección de entrega por defecto.
`id_ciudad`	INT4	No	FK → ciudad	Ciudad donde vive.
`password_hash`	TEXT	Sí	—	Contraseña cifrada; obligatoria para el administrador.
`rol`	VARCHAR(20)	No	—	'admin' o 'cliente'; por defecto 'cliente'.

categoria

Campo	Tipo	Nulo	Llave	Descripción
`id_categoria`	SERIAL	No	PK	Número de la categoría; lo asigna la base.
`nombre`	VARCHAR(80)	No	UNIQUE	Nombre de la categoría, sin repetir.
`descripcion`	VARCHAR(255)	Sí	—	Descripción opcional.

articulo

Campo	Tipo	Nulo	Llave	Descripción
`id_articulo`	SERIAL	No	PK	Número del artículo; lo asigna la base.
`nombre`	VARCHAR(120)	No	—	Nombre del artículo.
`descripcion`	VARCHAR(255)	Sí	—	Descripción opcional.
`precio`	NUMERIC(12,2)	No	—	Precio de venta en pesos; mayor que 0.
`stock`	INT4	No	—	Unidades disponibles; 0 o más, por defecto 0. Lo descuenta el disparador al vender.
`id_categoria`	INT4	No	FK → categoria	Categoría del artículo.

articulo_imagen

Campo	Tipo	Nulo	Llave	Descripción
`id_articulo`	INT4	No	PK, FK → articulo	Artículo dueño de la foto.
`tipo`	VARCHAR(20)	No	—	Formato: image/jpeg, image/png o image/webp.
`datos`	BYTEA	No	—	Bytes de la imagen, de 1 byte a 2 MB.
`actualizada`	TIMESTAMP	No	—	Fecha y hora en que se subió; por defecto, el momento actual.

venta

Campo	Tipo	Nulo	Llave	Descripción
id_venta	SERIAL	No	PK	Número de la venta o pedido.
fecha	DATE	No	—	Fecha del pedido; por defecto, hoy.
estado	VARCHAR(30)	No	—	Pendiente, Pagada, Enviada, Entregada o Cancelada; por defecto Pendiente.
direccion_envio	VARCHAR(150)	No	—	Dirección a la que se envía el pedido.
id_cliente	INT4	No	FK → cliente	Cliente que compró.

detalle_venta

Campo	Tipo	Nulo	Llave	Descripción
id_venta	INT4	No	PK, FK → venta	Venta a la que pertenece la línea.
id_articulo	INT4	No	PK, FK → articulo	Artículo vendido.
cantidad	INT4	No	—	Unidades vendidas; mayor que 0.

forma_pago

Campo	Tipo	Nulo	Llave	Descripción
id_pago	SERIAL	No	PK	Número del pago; lo asigna la base.
tipo	VARCHAR(40)	No	—	PSE, Tarjeta Visa, Tarjeta Mastercard, Efecty o Transferencia Bancaria.
monto	NUMERIC(12,2)	No	—	Valor pagado; mayor que 0.
fecha_pago	DATE	No	—	Fecha del pago; por defecto, hoy. No puede ser anterior a la fecha de la venta.
estado	VARCHAR(30)	No	—	Pendiente, Aprobado o Rechazado; por defecto Aprobado.
id_venta	INT4	No	FK → venta	Venta que se paga.

6. Código fuente

La aplicación está en PHP sin frameworks y la base en PostgreSQL. En `ventas_app/` están las páginas del superusuario (inicio, ventas, pagos, artículos, categorías, clientes y ciudades) y las de la tienda del cliente (catálogo, ficha, carrito, mis compras y mis datos); `ventas_app/includes/` tiene las funciones que usan todas (conexión, sesión, roles y validaciones). Las tablas y los disparadores están en `ventas_internet.sql`. Estas son las partes más importantes:

6.1 Base de datos: disparadores

Descontar el stock · `ventas_internet.sql`, líneas 112 a 142

Antes de guardar cada artículo de una venta revisa el inventario: si no alcanza, rechaza la operación; si alcanza, lo descuenta.

```
CREATE OR REPLACE FUNCTION fn_descontar_stock_venta()
RETURNS TRIGGER AS $$
DECLARE
    v_stock INT4;
BEGIN
    SELECT stock INTO v_stock
    FROM articulo
    WHERE id_articulo = NEW.id_articulo
    FOR UPDATE;

    IF v_stock IS NULL THEN
        RAISE EXCEPTION 'El artículo % no existe',
NEW.id_articulo;
    END IF;

    IF v_stock < NEW.cantidad THEN
        RAISE EXCEPTION 'Stock insuficiente para el
artículo %: disponible %, solicitado %',
            NEW.id_articulo, v_stock, NEW.cantidad;
    END IF;

    UPDATE articulo
    SET stock = stock - NEW.cantidad
    WHERE id_articulo = NEW.id_articulo;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_descontar_stock_venta
BEFORE INSERT ON detalle_venta
FOR EACH ROW
EXECUTE FUNCTION fn_descontar_stock_venta();
```

Validar la fecha del pago · `ventas_internet.sql`, líneas 145 a 166

Es la restricción de fecha inicial y fecha final: un pago no puede tener fecha anterior a la de su venta.

```
CREATE OR REPLACE FUNCTION fn_validar_fecha_pago()
RETURNS TRIGGER AS $$
DECLARE
    v_fecha_venta DATE;
BEGIN
    SELECT fecha INTO v_fecha_venta
    FROM venta
    WHERE id_venta = NEW.id_venta;

    IF NEW.fecha_pago < v_fecha_venta THEN
        RAISE EXCEPTION 'La fecha de pago (%) no puede
ser anterior a la fecha de la venta (%)',
            NEW.fecha_pago, v_fecha_venta;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_fecha_pago
BEFORE INSERT OR UPDATE ON forma_pago
FOR EACH ROW
EXECUTE FUNCTION fn_validar_fecha_pago();
```

6.2 Superusuario (administrador)

Agregar un artículo a una venta · `ventas_app/venta.php`, líneas 58 a 82

Valida los datos y hace el INSERT en detalle_venta; el disparador descuenta el stock o devuelve el error, que se muestra en la página.

```
// Agregar un artículo
if ($accion === 'agregar') {
    $id_articulo = post('id_articulo');
    $cantidad    = post('cantidad');

    if (!$editable) {
        $errores[] = 'Solo se pueden agregar artículos a una venta Pendiente.';
    }
    if (!fila('SELECT 1 FROM articulo WHERE id_articulo = ?', array((int)$id_articulo))) {
        $errores[] = 'Seleccione un artículo.';
    }
    validar_entero($cantidad, 'Cantidad', $errores, 1, 100000);

    if (!$errores) {
        try {
            // El disparador descuenta el stock o rechaza la línea
            consulta('INSERT INTO detalle_venta (id_venta, id_articulo, cantidad) VALUES (?, ?, ?)',
                     array($id, (int)$id_articulo, (int)$cantidad));
            flash('ok', 'Artículo agregado y stock descontado.');
            redirigir($aqui);
        } catch (PDOException $e) {
            $errores[] = mensaje_error_bd($e);
        }
    }
}
```

6.3 Cliente (tienda)

Confirmar el pedido del carrito · `ventas_app/carrito.php`, líneas 112 a 127

Crea en una sola transacción la venta, sus artículos y el pago pendiente. Si un artículo no tiene stock, el disparador lanza un error y no se guarda nada.

```
$venta = en_transaccion(function () use ($carrito, $direccion_envio, $tipo_pago) {
    $hoy = date('Y-m-d');
    $id_venta = (int)valor(
        "INSERT INTO venta (fecha, estado, direccion_envio, id_cliente) VALUES (?, 'Pendiente', ?, ?) RETURNING
id_venta",
        array($hoy, $direccion_envio, (int)$_SESSION['id_cliente'])
    );
    foreach ($carrito as $id_art => $cant) {
        // Si no hay stock, el disparador lanza un error y se deshace todo
        consulta('INSERT INTO detalle_venta (id_venta, id_articulo, cantidad) VALUES (?, ?, ?)',
                 array($id_venta, (int)$id_art, (int)$cant));
    }
    $venta = buscar_venta($id_venta);
    consulta("INSERT INTO forma_pago (tipo, monto, fecha_pago, estado, id_venta) VALUES (?, ?, ?, 'Pendiente',
?)",
             array($tipo_pago, $venta['total'], $hoy, $id_venta));
    return $venta;
});
```

6.4 Seguridad: rol de PostgreSQL según el usuario

aplicar_rol_bd() · `ventas_app/includes/auth.php`, líneas 37 a 46

Antes de consultar, la conexión toma el rol del usuario (administrador, cliente o visitante). Así la base, con sus permisos y la seguridad por filas, decide qué puede ver y cambiar cada uno.

```
// Cambia el rol de PostgreSQL según quién usa la página
function aplicar_rol_bd($rol = null)
{
    $rol = $rol ?? ($_SESSION['rol'] ?? 'visitante');
    $id_cliente = ($rol === 'cliente') ? (string)(int)($_SESSION['id_cliente'] ?? 0) : '';

    $roles_bd = array('admin' => 'rol_admin', 'cliente' => 'rol_cliente');
    db()->exec('SET ROLE ' . ($roles_bd[$rol] ?? 'rol_visitante'));
    consulta("SELECT set_config('app.id_cliente', ?, false)", array($id_cliente === '0' ? '' : $id_cliente));
}
```